

Porting from CSP vs. to CSMP++



CSMP workshop in Garmisch Partenkirchen 2011

What changes?

Changes affecting users migrating from CSP to CSMP++

- **SuperGroup** now is **Model**, **Group** now is **Region**
- Much of SuperGroup functionality has been transferred to **ModelSubDomain**. It acts as base class of **Region**, **Boundary** and **SplitBoundary**
- A distinction is now made between *unique*- and *non-unique* (potentially overlapping) Regions
- **Face** and **InterFace** class make up Boundaries and SplitBoundaries
- No longer a central **MemoryManager** exists. In stead, a distributed property storage is implemented via **LocalVariableStorage** classes that form part of the interfaces of the node, element etc.
- All node, element and any other numbering runs from 0..n-1 not 1..n
- Stand-alone ConstraintPoints are gone; Use Read(i, index) to retrieve corresponding INTEGRATION_POINT variables from the Element
- New variable placements and corresponding PLACEMENT flags: INTEGRATION_POINT, REGION, BOUNDARY, FV integration points etc.
- New variable types ARRAY and FLAGGED_ARRAY
- New default flag for variables is ANY
- **PDE_Integrator** replaces **Algorithm** and is also suited for the integration of finite volume equations
- **Solver** now is a permanent member of the **PDE_Integrator** (DLL to object instance issues)
- Multiple instances of the SAMG DLL can be addressed via the **SAMG_Solver** interface
- triangular_finite_volumes/ library removed
- Don't use AtBoundary (SG_BOUNDARY) flags but high-level interfaces of Model replacing some of this functionality
- completely new utilities in that directory

How to port CSP code to CSMP++

Basics

- namespace csp → csmc
- csp_float → double64
- stl_index → size_t
- Template parameters <fT,dim> change to <dim>
- SuperGroup → Model
- Group<csp_float,dim> → Region<dim,Element>
- GroupMonitor<> → RegionMonitor
- Algorithm<csp_float,dim> → PDE_Integrator<dim,Face,Boundary>
- e.Read(MemoryManager, prop_key.index, var);
→ e.Read(prop_key, var);
- read/store ConstraintPoint variables: → e.Read(i, prop_key, var);
- e.Status(vec_key); → e.Status(vec_key, i);
- CSP_ERROR → ERROR (watch wingdi.h header)
- SG_BOUNDARY → BOX_BOUNDARY
- some classes are not templates anymore: Matrix, ScalarVariable etc.

How to port Element stuff (2)

Node<>* **Node**(i); ➔ Node<>* **N**(i);

stl_index ID(); ➔ size_t Idx();

Finite Element / Finite Volume functionality:

- FiniteElementTraits<size_t,template<size_t> class>
- FiniteVolumeTraits<size_t,template<size_t> class>
- ElementGeometryTraits<size_t,template<size_t> class>

User friendly names, hiding the implementation:

```
typedef ElementGeometryTraits<3U,Element> GeometryTraits;
```

```
GeometryTraits gt; ...
```

Finite Volume stuff

Replace:

- FiniteVolumeManager
- FiniteVolumeFluidPhasePropagation
- NodalFluidVolumeSource
- BROOKS_COREY etc.

with:

- NodeCenteredFiniteVolumeTransport and derived classes
- FiniteElementConnectivity no longer exists
- 2-phase flow header file with TwoPhaseModel

Miscellaneous

New Singletons (see Philipp's presentation)

- `global_time` → `model_time`
- instance of `ErrorHandler` `skm_err` → `csmp_error`

Advanced ported / portable examples: 10 published CSMP applications

Find in example_suite/ and test_suite/ directories:

1. Well testing
2. Flow models from the pore to the field scale
 1. Navier-Stokes flow in porous media
 2. DFM models of Naturally Fractured Reservoirs (NFR)
 3. NFR sector-scale modelling with upscaled relative permeabilities
3. Mechanical modelling
4. Equations of State
5. Heat flow
6. Hydrothermal circulation at mid ocean ridges
7. Formation of the Mt. Isa giant copper deposit
8. Reactive transport
9. Tracer tests
10. Embedded discontinuities

User-affecting changes compared with version 5.

- numberings all from 0..n-1
- stl_index -> size_t
- (issue:
 - **for (typename vector<Node<fT,dim>*>::size_type
i=0U; i<(*it)->Nodes(); i++))**
 - *but do not use loops by numbers if you can*
- csp_float -> double64
- Read(), Store(), Status() -> no more PropertyStorage arguments
 - **const fT satw = (*nit).Read(sg.ReferencePropertyStorage(),
sw_key.index); -> Read(sw_key);**
- Region takes over many tasks of SuperGroup
- Region variables
- Exception handling
- Model interface:
 - shorter names for accessors
 - ReferencePropertyDatabase() -> Database() etc.
 - **model.InputPropertyValue("porosity", makeScalar(PLAIN,0.1));**

continued

- Boundary conditions
 - for box-shaped models boundary Regions (Boundaries) are retained in mesh but they are not included into Region “Model”
 - now these are used only on request to compute conditions
- Assigning constraints:
 - **gref.Node(0U).Store(model.Database().StorageKey("fluid pressure"), pf_top);**
 - **model.AssignConstraint("fluid pressure", model.Nodes(), pf_top);**

Assigning variable values and boundary conditions

- `InputPropertyValue( "fluid pressure", makeScalar( DIRICH, 5e6 ) );`
- How to apply Dirichlet condition:
- `InputPropertyValue( "fluid pressure", makeScalar( DIRICH, 5e6 ) );`

```
template<class T>
void InputBoundaryValue( BOX_BOUNDARY, const char* input_prop, const T& value );

void InputBoundaryFlags( BOX_BOUNDARY, const char* property, const std::vector<VARIABLE_FLAG>&
flags );

void InterpolateBoundaryValues( BOX_BOUNDARY, const char* input_prop, const
std::vector<ScalarVariable> &bvalues );

void InterpolateBoundaryValues( BOX_BOUNDARY, const char* input_prop, const
std::vector<VectorVariable<dim> >&bvalues );
```

looping over the nodes of a SuperGroup, now Model

old way

- **for (typename deque<Node<fT,dim> >::const_iterator
nit=sg.Mesh().NodesBegin(); nit!
=sg.MeshManager().NodesEnd(); nit++) {const fT sato =
(*nit).Read(sg.ReferencePropertyStorage(), so_key .index)**

new way

- **const Region<dim,Element>& sgref = sg.Region("Model");
for (typename vector<Node<dim>*>::const_iterator
nit=sgref.NodesBegin(); nit!=sgref.NodesEnd(); nit++)
{ const double64 sato = (*nit)->Read(so_key);**

Accessing parent elements from Nodes

Now:

- **size_t Parents() const; // parent elements of node**
Element<dim>* Parent(size_t) const; size_t
ParentNodeNumber(size_t parent_element) const;

Formerly:

- **stl_index Parents() const; stl_index**
ParentLocalNodeNumber(stl_index) const; stl_index
ParentElementID(stl_index) const;

Accessing IntegrationPoint variables

- now for Element, Face, or InterFace:
 - **template<class T> void Read(size_t, const csmpt::Index&, T&) const;**
 - **template<class T> void Store(size_t, const csmpt::Index&, const T&);**

formerly:

```
ConstraintPoint<fT,dim>*  
  ConnectedConstraintPoint( stl_index n ) const;  
->Read(...);
```

unique vs. non-unique Regions

- vectors vs. maps?

Porting issues

- make sure that all the carefully written main files are working again.
- **Sebastian: All old papers are reproduced as CSMP++ test cases.**
- **Shared repository of papers alongside with the tests**

Testing Needs

- FiniteElement tests automated?

HW SVN fixes to be incorporated

- DispersivityVisitor::Visit(Element)
- ExplicitNodeCenteredFiniteVolumeTransport::ComposeSolution() – source term needs to be multiplied by time_increment
- Check-in Robert's CO2 equation of state
- Additional option in SAMG.h to use it as its own preconditioner
- Bug: ElementFiniteVolumeTraits
- additional functionality ComputationalSettings
- SAMG_Solver – Yan, eliminating Dirichlet constraints (serial version only), point-based approach
- Sebastian: Matrix methods to get norms for matrices (Dany/Sebastian) – incomplete LU decomposition (within SAMG)
- Sebastian's Corey model with Endpoints
- Max fractional flow value?
- FluxBalance function now public NCVT
- ParallelExplicitFiniteVolume scheme
- *Sebastian will go over the code and make changes*

Other issues

- Multiple variables in computation -> whole matrix is transposed by accident
- Matrix - copy constructor does not resize matrix correctly
- Models with lower dimensional elements, computation of gravity term have to be considered correctly
- `cout/clog/cerr` `freopen` statement to dump all output away
- `NodeCenteredFiniteVolumeTransport` – does not work for solute transport (get Model related version)
- `ExplicitNodeCenteredFiniteVolumeTransport` – does not work
- Porosity – scaling of courant increment in CFL calculation: Is it needed?
- Don't scale porosity with thickness anymore, but use explicit thickness attribute like aperture do achieve the desired result.

PropertyHandle

- provide simple operations: interpolate to barycentre, interpolate to integrationpoint, averages: arithmetic, harmonic, geometric
- binary operators
- restriction to regions
- implementation using visitors
- framework : result variable, input variable etc.
- Maybe the result propertyhandle needs to be flagged as such

PDE Operators

- A decision is needed what to do with the surface integrals
- Also we need to tell the PDE_Integrator which boundaries shall be included in the computation to account for surface integrals.
- Decision: Keep surface integrals to tell PDE_Int that it needs to go over the boundaries. PDE_Int will then go over all boundaries to detect by itself where these integral are non zero.

Remaining Actions

- Define how Boundary and SplitBoundary are to be handled by PDE_Integrator and other classes
- create test cases and examples how PDE_Integrator works on the new model boundaries
- check new PDE_Integrator
- IGet SplitBoundary to run (SKM):
 - The methods CreateFor() etc. must be implemented tested
 - Creation of SplitBoudaries involves insertion of new nodes etc.
 - fix supporting Face and InterFace functionality

To do:

- Freeze interface (including pending modifications)
- Fix bugs in LocalVariableStorage (including array storage, contact Jens Reindl)
- Incorporate Shaho's modification of TensorVariable with less flags
- Decision about Status() method flag access
- Shaho + Ali will build a GUI-driven reservoir simulator (extra funding from Pavle Matisek)
- SKM: fix surface integral pde operators (code added to PDE_Integrator + refactoring of surface integrals): options: box boundaries vs. all boundaries in a model
- Check whether lower-dimensional fractures shall be represented by boundaries
- SKM: extend the PropertyHandle to include interpolation, extrapolation and averaging
- ExplicitFiniteVolumeTransportAlgorithm -> get second order scheme to work
- Remove any reference to ConstraintPoints

Old to dos: contributions promised by Leoben team

- SKM
 - **fix Face and InterFace functionality**
 - **complete Boundary and SplitBoundary functionality**
 - create test cases and examples how PDE_Integrator works on the new model boundaries
 - **check new PDE_Integrator**
- Julian & SKM
 - Fix single-phase transport scheme and unit test it
 - Use Factory pattern to create FiniteElements on the fly (cleanup issue with 2D and 3D versions of surface elements)
 - Illustration of CSMP FEM and FVM frameworks (interpolation functions, derivatives, integration points, PDE operators – use quadratic FEM example from UG to explain how a FEM works)
 - Read HW wiki blog to enlist and fix all issues that were detected after new CSMP++ branched-off from old CSP
- Siroos – check basic examples (Ali needs to get CSMP running on Siroos computer)
- Matthias
 - Petrel interface – demo and explanations
- Ali
 - Create pipelines for sequential operations
 - GUI for reservoir simulator
- Philipp & SKM
 - Complete and test examples
 - **Introduce policies**

Pending Actions from 2010

1. source_code/CSMP_DEBUG_SCHEDULE.h: replace by issue-tracker
2. On this workshop:
 1. Get rid of Interrelation and interrelation/ library
 2. move functionality to Visitor and PropertyHandle, Visitor functionality: distinction between input and output variables, generation of variables
3. Build generic component and phase-based transport scheme (Sebastian & Thomas)
4. Create PDE operators to be able to do FV calculations (Christine & Sebastian, within one year, something simpler asap)
5. support compile-time variables (SKM)
6. Add new variable type ARRAY composite, LocalVariableStorage, IntegrationPointVariableStorage, PLACEMENT (Leoben)
7. IntegrationPoint... everywhere, ConstraintPoint becomes IntegrationPoint (INTEGRATION_POINT)
8. Sort out point-based computations (SG creates example for coupled soln.)
9. Add assumptions / restrictions / consequences section to DOxygen documentation of classes (TD identifies classes that need this)
10. Split DATA_STYLE enum into NEUMANN (constant gradient) and CONSTANT_FLUX (SKM) -> change DATA_STYLE into **VARIABLE_FLAG**.

More Actions

- Constants file, unit conversions, strict definitions of constants (TD + Ali), hardwired constants, these should use new module.
- AdvectionDiffusion**Integrator** (small-Peclet number only) SKM (put check that Peclet number is not too big)
- SteadyStateDiffusion**Integrator** & TransientDiffusion**Integrator** (SKM)
- eos/ directory: sort out and provide tests / examples establishing how the capabilities should be used (ETH group). New stuff added from HW svn.
- CompressedRowMatrix (Julian) – base class and more specific subclass, unit test
- Get SplitBoundary to run
- In any part of the code, look for hardwired variables. If found, think about how this code could be made more generic.
- VisitorManager for use with PropertyHandle

More Actions

- SKM get Adriana to put new functionality into SVN
- Jacobian functions do not need to be virtual
- Profiling (Julian)
- Two phase models should not include densities and or viscosities?
- Everybody writes one Unit Test and submits it to Julian until the end of Octobre.
- There must be a release version of the new CSMP (optimised compiled version of the repository code)
- Typedefs are still badly needed to better document parts of the code
- Ali to release model configurator
- Code to eliminate elements that are degenerate (non-manifold vertices) from the computations.

Discussions

- Variable types:
 - templated Index inherited from Index, can give speed-ups if used, use is only optional (to do this efficiently LocalVariableStorage must be a policy class.
- Interrelation: problem: alphabetically ordered Operands cause bugs. Order should be the one from the equation you write (expression templates)
- Operand is written in a very clumsy way.
- Qualify in the Interrelation which way the variables are interpolated.
- SAMG solver 1-5